

Message queue

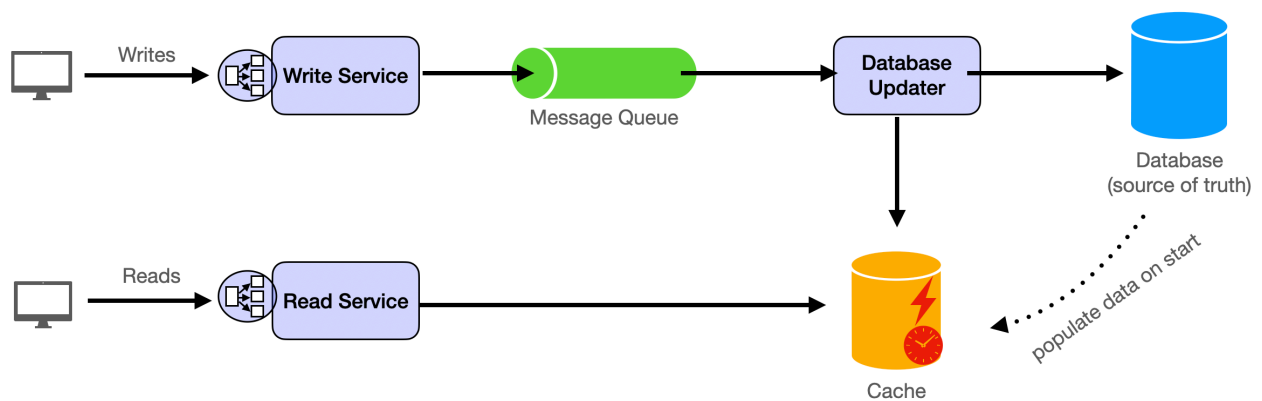
 systemdesignschool.io/fundamentals/system-design-template



Design Template

The Template

Here is the common template to design scalable services (and therefore solve many system design problems):



A very high-level takeaway:

- Write to message queue
- Read from cache

Let's take a look at it step-by-step.

Design Template: Component Breakdown

Stateless Services

- Scalable and stateless, these services can be expanded by adding new machines and integrating them through a load balancer.
- **Write Service:**
Receives client requests and forwards them to the message queue.
- **Read Service:**
Handles read requests from clients by accessing the cache.

Database

Serves as cold storage and the source of truth. However, we do not normally read directly from the database since it can be quite slow when the request volume is high.

Message Queue

- A buffer between writer services and data storage.
- **Producers:**
Comprised of write services that send data changes to the queue.
- **Consumers:**
 - Involved in updating both the database and the cache.
 - **Database Updater:**
Asynchronous workers update the database by retrieving jobs from the message queue.
 - **Cache Updater:**
Asynchronous workers refresh the cache by fetching jobs from the message queue.

Cache

Facilitates fast and efficient read operations.

Now let's take a look in detail.

Dataflow path

Almost all applications can be broken down into read requests and write requests.

Because read and write have completely different implications (read doesn't mutate; write mutates database), we discuss write path and read path separately.

Read path

For modern large-scale applications with millions of daily users, we almost always read from cache instead of from the database directly. The database acts as a permanent storage solution. Asynchronous jobs frequently transfer data from the database to the cache.

Write path

Write requests are pushed into a /fundamentals/message-queue, allowing backend workers to manage the writing process. This approach balances the processing speeds of different system components, offering a responsive user experience.

Message queue is essential to scaling out our system to handle write requests.

- Producers: Insert messages into the queue.
- Consumers: Retrieve and process messages asynchronously.

The necessity of message queues arises from:

- Varying Processing Rates: Producers and consumers handle data at different speeds, necessitating a buffer.
 - e.g., the frontend posts comments a lot faster than the backend can write to the db
 - e.g., the frontend booking request is a lot faster than the backend booking service (needs to contact 3rd party)
- Fault Tolerance: They ensure the persistence of messages, preventing data loss during failures.
 - imagine having the write service calling the db updater (workers) directly. If the request fails, the request is lost
 - with mq, messages are persisted in the queue so if workers are down, messages remain safe and can be processed when workers recover

One database per service

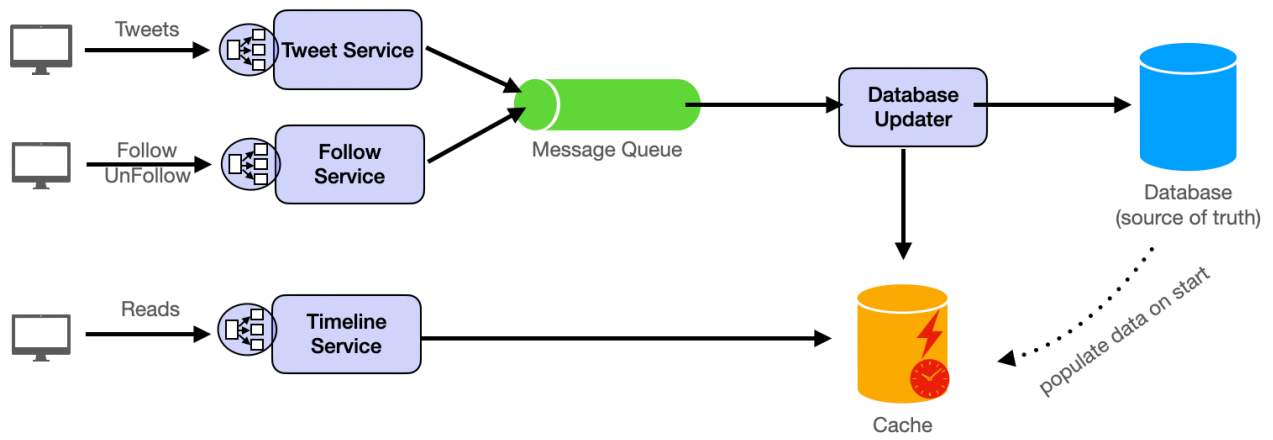
One additional design recommendation I would like to mention is to not expose the database directly to clients. You almost always create a service that goes with the database and expose the data through APIs.

Using the Template

Now let's take a look at a couple of concrete examples.

1. [Design Twitter](#)

Twitter's timeline is a classic template example. Tweets in a user's timeline are pre-fetched for each user into an "inbox" inside the cache and displayed to the user when they open the app on their devices.



2. Design WhatsApp

WhatsApp utilizes the design template with a slight modification. When a user is online, messages are directly transmitted to their web socket server for delivery. Conversely, if the user is offline, the messages are stored in their "inbox" within the cache.

